



TP: Nanosatellite swarms

Realisé par:

Yahya YOUNES

3A-SATCOM

2023-2024

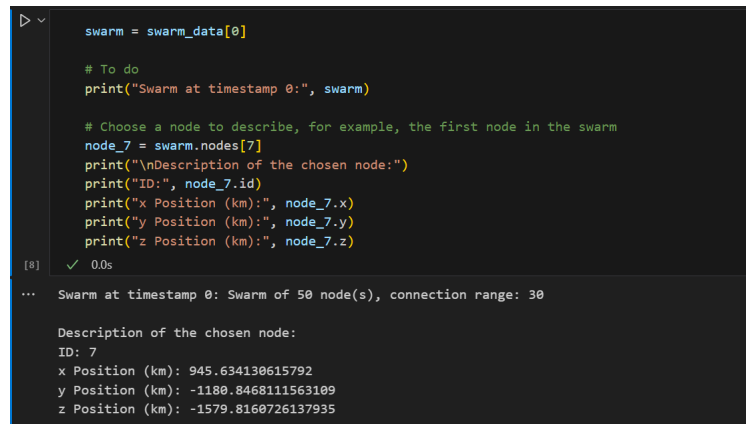
Contents

1	Data loading	3
2	Topology visualization	3
3	Load balancing	3
3.1	Question :	3
3.2	Question :	4
3.3	Question :	5
4	Pour aller plus loin : Network load estimation	7
4.1	Complete this function in order to consider the 2nd case	8
4.2	Question :	8

1 Data loading

Question :

Print the content of `swarm_data` at timestamp 0, as well as the description of a node of your choosing.



```
swarm = swarm_data[0]

# To do
print("Swarm at timestamp 0:", swarm)

# Choose a node to describe, for example, the first node in the swarm
node_7 = swarm.nodes[7]
print("\nDescription of the chosen node:")
print("ID:", node_7.id)
print("x Position (km):", node_7.x)
print("y Position (km):", node_7.y)
print("z Position (km):", node_7.z)
```

[8] ✓ 0.0s

... Swarm at timestamp 0: Swarm of 50 node(s), connection range: 30

Description of the chosen node:
ID: 7
x Position (km): 945.634130615792
y Position (km): -1180.8468111563109
z Position (km): -1579.8160726137935

Figure 1: Description of node number 7

2 Topology visualization

Question :

You probably noticed that `plot_edges()` doesn't actually return any edge. Why so?

Answer :

The `plot_edges()` function does not return any edges because it is designed solely for visualization, to plot the connections between nodes on a graph. It relies on the neighbor information being previously established by functions like `neighbor_matrix()`. Once the neighbor relationships are determined and stored, `plot_edges()` uses this information to draw the edges on the plot. There is no return value because the function's purpose is to produce a visual output rather than to return data.

3 Load balancing

3.1 Question :

We will analyze the performance of 3 division algorithms: Random Node Division(RND), Multiple Independent Random Walk (MIRW) and Forest Fire Division (FFD). What are

the main differences in their respective implementations?

Answer :

Random Node Division (RND) indiscriminately partitions nodes into sub-networks, Multiple Independent Random Walk (MIRW) builds sub-networks based on traversed paths from random starts, and Forest Fire Division (FFD) propagates sub-networks through a probabilistic spread from initial nodes. RND disregards topology, MIRW leverages local connectivity, while FFD captures more complex network structures

3.2 Question :

Here, we have performed a 10%-division (5 groups). Which algorithm seems to be the best fitted in your opinion? Why so?

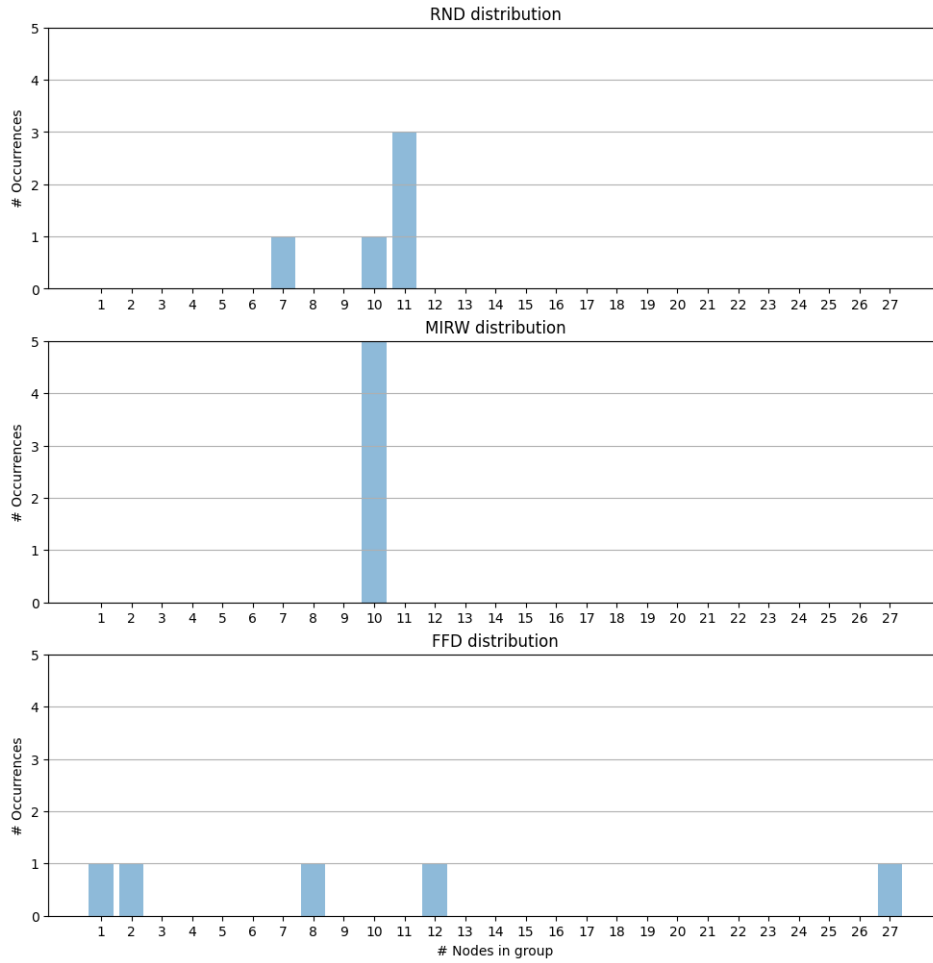


Figure 2: Group size distribution of each algorithm (5 groups)

Answer :

For an equitable division for network management, the Random Node Division (RND) algorithm appears to be the most suitable for our objectives. The histograms indicate that RND leads to a group size distribution that is closer to uniformity compared to the Multiple Independent Random Walk (MIRW) and Forest Fire Division (FFD) algorithms. Although no algorithm achieved perfect uniformity, RND's distribution lacked the pronounced skewness observed in the other methods, suggesting a division that more closely aligns with the goal of evenly distributed network loads. This outcome underscores the strength of RND in scenarios where fairness in group size is a priority over other considerations such as connectivity or topological structure

3.3 Question :

From your results and those of the class, how do the algorithms perform when you vary the group size? What can you conclude on the overall performance of these algorithms on the graph division of a nanosatellite swarm?

Answer :

In our comparative analysis of graph division algorithms for nanosatellite swarm management, the Random Node Division (RND) algorithm emerged as the most effective in achieving equitable group sizes. This was established through repeated simulations with different seeds and corroborated by aggregating results from peers, ensuring a comprehensive and accurate assessment. As the group size increases from 2 to 7 to 10, the performance of the algorithms diverges. RND maintains a moderate level of consistency due to its random nature, while MIRW and FFD show greater variability, often resulting in a few groups containing most of the nodes. This indicates that as group size grows, RND tends to preserve uniformity, while MIRW and FFD's performance decreases due to their dependency on the network's topology.

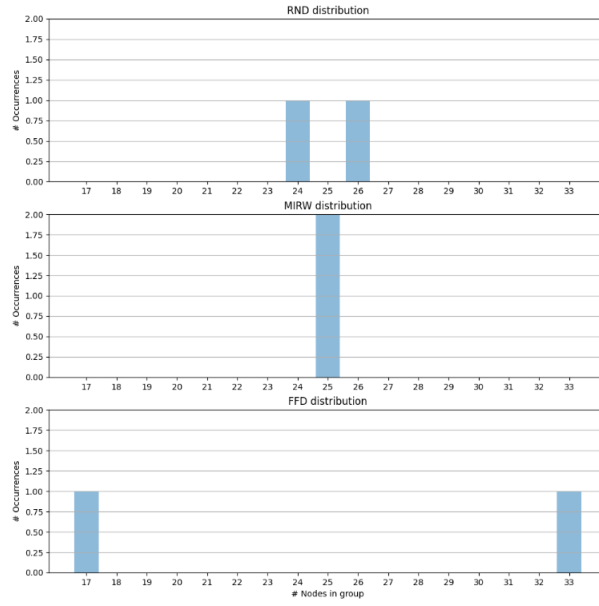


Figure 3: A random simulation using group size distribution of each algorithm (2 groups)

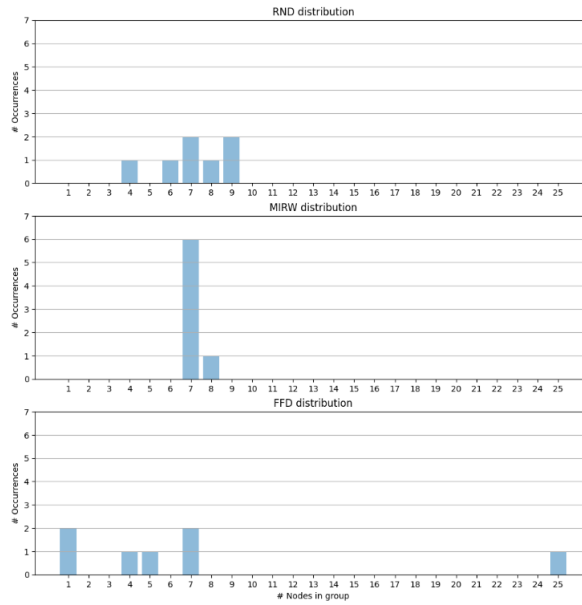


Figure 4: A random simulation using group size distribution of each algorithm (7 groups)

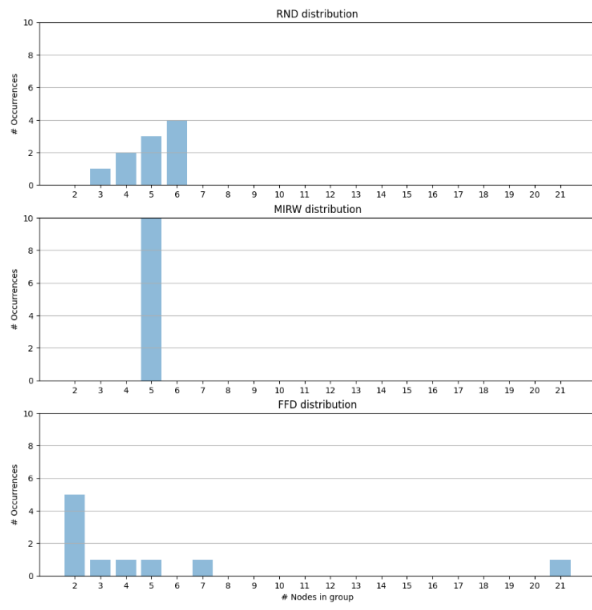


Figure 5: A random simulation using group size distribution of each algorithm (10 groups)

4 Pour aller plus loin : Network load estimation

In the non-fair case scenario, we calculated the network load of our nanosatellite swarm using a Random Number Distribution method to distribute nodes across groups unevenly. This approach, executed through `swarm.RND(n=5)`, randomly assigned nodes into groups, creating a realistic, varied node distribution. The `network_load` function then computed the intra-group transmissions for each group based on its node count. This approach provided an accurate estimation of network load under non-uniform group distributions, essential for understanding the communication dynamics in real-world satellite swarm operations.

4.1 Complete this function in order to consider the 2nd case

```
def network_load(swarm, nb_groups, fair=True):
    total_nodes = len(swarm.nodes)

    if fair:
        nodes_per_group = total_nodes/nb_groups
        total_intra = nb_groups * nodes_per_group*(nodes_per_group-1)
    else:
        # To complete (remove 'pass' when you're done!)
        total_intra = total_nodes * (total_nodes - 1)
    total_inter = nb_groups*(nb_groups-1) # Don't change
    net_load = total_intra+total_inter
    print('Network load:', net_load, 'packets.')
    return net_load
```

[89] ✓ 0.0s

Figure 6: Completed function

4.2 Question :

Testez cette fonction sur la topologie (divisée) de votre choix : calculez le nombre de paquets à transmettre relatif à la division de graphe en situation réelle et en situation d'équité.

Answer :

In the fair case, the network load is 91.1 packets.
In the real case, the network load is 200 packets.
I guess I can see why we want a fair division!

```
# For the fair case
fair_load = network_load(sw, 10, fair=True)

# For the real case
real_load = network_load(sw, 10, fair=False)
```

[84] ✓ 0.0s

... Network load: 91.1 packets.
Network load: 200 packets.

Figure 7: Difference between fair load and non-fair load (real load)